

Between knowing-how and knowing-that
or
An experimental journey of “teaching” a “human” to “swim”

Git Repository: <https://github.com/jkohler-u/Swimming_agent/tree/main>

Malin Braatz and Judith Kohler

July 21, 2026

Contributions:

- Idea Brainstorming + Literature Research + Method Research: Judith, Malin
- overall, the project was a team effort and both were involved in all sections. Main Coding effort for individual sections:
 - RL diving worm: Judith
 - RL breathing worm: Malin
 - RL human: Judith
 - Control for LLM approach: Malin
 - LLM integration + prompt engineering: Judith
- Report Writing: Judith, Malin

Declaration of AI Use

Over the course of this project, we utilized LLMs for assistance with coding, generating search prompts when looking for literature, and rephrasing sections of the report. Subsequently, we thoroughly reviewed and edited the material as necessary and assume full responsibility for the content.

Abstract

In this report we are implementing two different control mechanisms to see how they perform at steering a humanoid swimmer. We first try a classical reinforcement learning (RL) approach utilizing PPO and optimizing the reward function. In our second trial we implement a control loop utilizing a PD controller and JCT to allow for an LLM to instruct the next action. For both trials, we then systematically test the impact of the reward function and the prompt, respectively, on their operation. Based on this, we then assess their performance and try to characterize their abilities.

1 Introduction

Our project mainly drew from two inspirations:

1. After frequently encountering the classical reinforcement learning (RL) example of teaching a humanoid to walk, we were curious to see how this would translate into other movement modalities
2. Inspired by the classic debate in Philosophy of Science about different forms of knowledge, we were curious to see how different learning agents would fare at this task

Both parts of the projects drew their unique challenges.

While there are many resources out there on teaching a humanoid RL agent to walk (Arjun, 2025; Singh et al., 2023) and teaching the classic MuJoCo Swimmer to swim (Farama Foundation, 2024), there is very little documentation around teaching a humanoid to swim. This meant that we had only basic references when setting up our training environment and our reward functions.

Conversely, the idea of involving an LLM in a control task is better explored. There are numerous angles to including an LLM into the control loop. It can take the role of a world model to break down tasks (Ahn et al., 2022), can be trained in place of an RL agent by reframing the learning objective to be transformer friendly (Chen et al., 2021), or utilized to set rewards (Yu et al., 2023).

Our goal is much more basic, as we do not aim to find the best possible application of an LLM in a control task, but rather want to explore the extent of their baseline capabilities. Most similar to our approach is the work by Wang et al. (2024), who also prompt an LLM to get actions without prior finetuning for the task. The resulting policy allows a quadruped robot to walk on uneven terrain. Contrary to our approach, however, they provide the LLM with RL-generated actions in the prompt to seed it and allow the LLM to set the state of all joints while we only have the model set the new hand/foot positions.

2 Methodology

We developed a common simulation environment for humanoid swimming and evaluated two control paradigms: reinforcement learning and LLM-based control. This section describes the simulation environment, the reinforcement learning setup, and the architecture of the LLM controller.

All the code used to produce the results presented in this report can be found in our GitHub (<https://github.com/jkohler-u/Swimming-agent.git>).

2.1 Setting up the Environment

2.1.1 Implementing Water

To simulate water, we used MuJoCo’s fluid dynamics model based on ellipsoidal body approximations (MuJoCo, 2025). The model is not a full Computational fluid Dynamics (CFD) simulation. Instead, each rigid body is approximated as an ellipsoid, which allows for efficient computation of hydrodynamic forces and torques acting on the body as it moves through the fluid. The model accounts for buoyancy, drag, lift and added mass effects. This simplified representation of fluid dynamics is well suited for RL, which requires a large number of simulation steps during training. The fluid coefficients were configured as `fluidcoef="0.5 0.25 1.5 1.0 1.0"`, following MuJoCo’s parameterization of drag, lift, added-mass, and related fluid effects.

2.1.2 Implementing a Water surface

Unfortunately, MuJoCo’s fluid model only supports a single fluid medium for the entire simulation environment. Thus, it is not possible to model air and water simultaneously. To keep the simulation and computation simple, we chose to only visually display a water surface while applying the water dynamics uniformly throughout the entire environment.

2.2 Training the Reinforcement Agent

2.2.1 Proximal Policy Optimization

To train the RL agent, we use Proximal Policy Optimization (PPO) (Schulman et al., 2017). PPO is a policy-gradient RL algorithm that is widely used for locomotion tasks (Heess et al., 2017). It combines a comparatively simple implementation with stable training behaviour, making it well suited for continuous control problems (Schulman et al., 2017). During training, PPO optimizes a policy that maps observations of the environment to actions, with the aim of maximizing the expected cumulative reward.

PPO can be viewed as a simplified and computationally efficient approximation of trust-region policy optimization methods. Trust-region methods restrict the magnitude of each policy update such that the updated policy remains close to the previous one, which achieves a more stable learning behaviour.

2.2.2 Worm Setup



Figure 1: The two XML models we used - left: with head for the breathing task, right - basic model for the pure swimming trials

To ensure our ideas are plausible we started our exploration with a less complicated, worm-like model. First, to figure out the details of our water simulation, and later to experiment with how to best simulate the water surface. We used two different worm models (see Fig 1), adding a head and an additional segment for the worm, not only trained to perform swimming, but also keep its head above the water surface. Our final training setup looks as follows: We use Stable Baseline3’s implementation of PPO in a fairly standard configuration ($\text{lr} = 0.001$, $\text{batch_size} = 64$, $\text{n_steps} = 4048$), training for 2,000,000 steps in 8 environments. Aside from the positions and velocity, we added Euler angle conversion for better interpretability to the observations as well as a boolean to represent whether the head is above and the body is below the waterline.

2.2.3 Human Setup

When graduating to a human model we did so in three steps, iteratively increasing the degrees of freedom we are controlling (see Fig. 2). We first started with a simple torso with arms, later added legs, and finally moved on to the realistic humanoid model by Google DeepMind (2024), originally intended for walking simulations. We modified all XML files used according to the water simulation described previously. Our training setup is very similar to the worm setup. We use the same PPO setup and the same observed parameters, but increase the training steps to 2,500,000. We also developed the reward function further, which will be discussed in detail in the results.

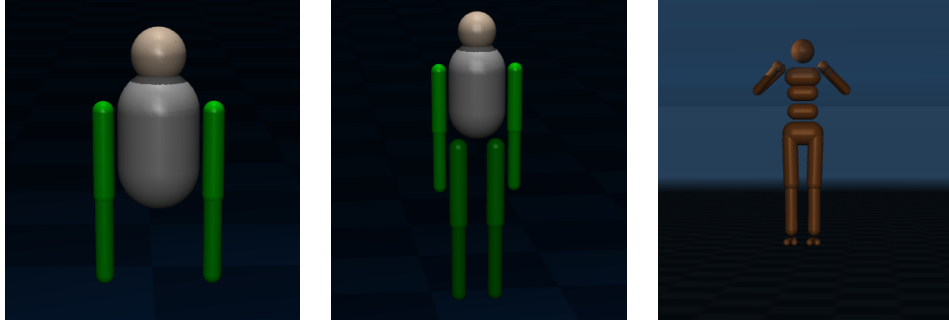


Figure 2: The three humanoid XML models we used, left to right with increasing degrees of freedom

2.3 Connecting the LLM to a controller

In the LLM modality we set out to prompt an LLM to correctly control a humanoid to swim. We settled for the backstroke as the swimming motion we aimed for, as the limb movements are comparably easy to describe without any visual aids. To simplify the task slightly, we modified the original XML file so the humanoid starts out laying on its back.

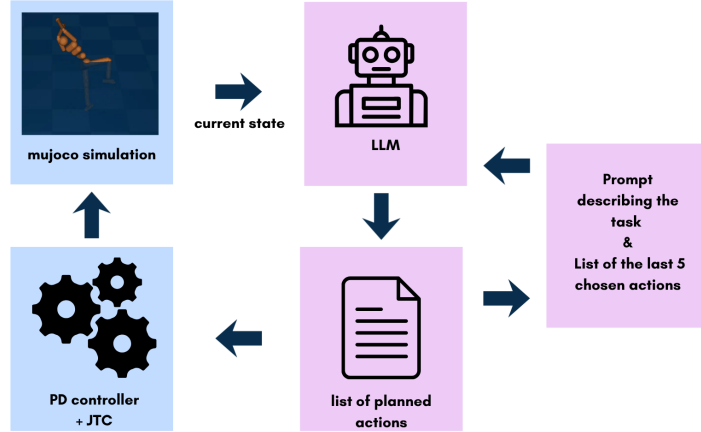


Figure 3: Enter Caption

To enable high-level control of the humanoid, we developed a two-part architecture, consisting of a high-level action-generation loop and a low-level control loop. Before integrating the LLM, we first validated the control loop independently. The control loop consists of a Cartesian proportional-derivative (PD) controller that moves the humanoid’s hands and feet to arbitrary target positions. Instead of directly controlling individual joint angles, desired limb positions are specified in a Cartesian coordinate system relative to the torso. This representation makes the controller independent of the humanoid’s global position and orientation, allowing the same movements to be executed regardless of the body’s pose in the world.

At every control step, the current position relative to the torso is computed for each controlled limb and compared with its desired target position. The resulting position error is used by the PD controller to generate a desired Cartesian force applied to the end effector. Using the torso-relative Jacobian, these Cartesian forces are transformed into joint torques through Jacobian Transpose Control (JTC). These joint torques are then converted into actuator control inputs and applied to the MuJoCo simulation.

Before integrating the LLM, the controller was validated using a procedural backstroke generator. The generator produced target positions following predefined elliptical trajectories for the arms and alternating flutter kicks for the legs. This experiment demonstrated that the controller was capable of producing backstroke-like swimming behavior. However, for larger arm radii, the humanoid had

a tendency to exhibit unstable rolling motions that caused the body to flip. See pictures of the performance in the Appendix (6).

In the next step the procedural trajectory generator was replaced by the LLM. The action-generation loop queries the LLM for the next action and then saves it in an action list. The prompt for the LLM contains:

- Setup: A description of the situation, how the humanoid can move, and how the LLM can control it
- Explanation: A detailed description of the Backstroke, depending on modality, including examples of how this might translate into coordinates
- An example of how to format the output
- A list of the last three actions and resulting velocities

Depending on the LLM used, we use either Ollama or the OpenAI API to query gemma4 or gpt4mini respectively. The LLM then returns the intended actions for each limb (hands/feet) of the agent relative to its torso in a JSON format. The intended actions are then added to the action list. The action list is the bottleneck in this control loop, as the LLM takes more time to generate actions than the controller takes to apply them.

3 Results

We evaluated both approaches with respect to two primary objectives:

- train the agent to perform a consistent swimming motion to achieve a considerable forward movement
- train the agent to keep its head above the water surface while swimming

3.1 Reinforcement Learning

3.1.1 Worm

Before training a humanoid swimmer, we validated our reward design on a simplified worm agent with substantially fewer degrees of freedom. After some exploration we converged to the set of components and baseline values for our reward function, which was designed to encourage forward swimming while maintaining a realistic swimming posture. We included a forward reward and velocity punishment to encourage forward motion. Furthermore, we added a continuous head reward, highest if the head is just above the water surface, and a body reward that rewards body segments below the water surface, with larger rewards for segments that are further below the surface. Additionally, a body punishment penalizes for the number of body segments above the water surface, and a smoothness term should suppress abrupt changes in the actions. To avoid undesirable behaviours, episodes are terminated, and a penalty is applied if the head remains submerged or the forward velocity stays below a minimum threshold for longer than a leniency period. The details of the reward function are depicted in table 1.

Parameter Name	Reward Contribution	Baseline Value
Forward reward	$forward_velocity \times scaling_value \times 10$	3
Head punishment	Applied if the head remains underwater longer than the leniency threshold	3
Velocity punishment	Applied if the forward velocity remains below 0.05 m/s longer than the leniency threshold	1
Smoothness reward	$-\sum(a_t - a_{t-1})^2 \times scaling_value$	0.2
Continuous head reward	If the head is just above the water surface: $(head_height - surface)^2 \times scaling_value$	2
Continuous body reward	If the body is below the water surface: $(surface - body_height)^2 \times scaling_value$	2
Continuous body punishment	For each body segment above the water surface: $(segment_height - surface - 0.5)^2 \times scaling_value$	1
Survival reward	Constant reward added every timestep	0.1

Table 1: Baseline reward parameters used for training the swimming worm. In addition, a leniency parameter is used that specifies how many consecutive timesteps the agent may remain underwater or below the minimum forward velocity before the episode is terminated and the corresponding penalty is applied.

With the baseline values for the reward function we achieved the following performance, measured over 1000 steps (see 4):

Metric	Value
Average Forward Velocity	0.95
Head Above Water:	73.9%
Longest streak of head below water	26 steps

Table 2: Our metrics applied to the worm, with baseline parameters

These metrics indicate that the learned policy was generally capable of learning to move in water while eventually keeping the head above the water. The video of the evaluation shows that the worm first curls up and turns around. It then swims tail-first, performing rhythmic kicking motions with its tail while keeping most of its body underwater. During this motion, the head alternates between being submerged and above the water surface while remaining oriented upwards. Time series pictures of the swimming motion are in the appendix (App. A).

To evaluate the contribution of the individual reward components, we systematically tested their influence: starting from the baseline reward function, we trained an additional agent for each reward component and set them to zero while keeping all other parameters unchanged (see Tab. 3).

The forward reward proved to be the most important component, as removing it nearly eliminated forward motion. Removing the continuous body punishment, caused the worm to eventually leave the water and fly, which shows that the reward achieves its function of keeping the main swimming motion in water. Removing the continuous head reward resulted in an upright and curled-up posture of the worm, with the body underwater, while the head remained well above the surface at the beginning of the episode. This suggests that the reward helps the agent maintain a more natural swimming posture with the head close to the water surface. This condition and the condition of removing the continuous body reward resulted in early episode termination because the agent eventually drowned (after 734 steps for the continuous head reward and after 167 steps for the continuous body reward). This is another sign that both reward components are essential for a beneficial swimming posture. Furthermore, the survival reward seems to be effective for sustaining swimming behaviour throughout the episode. Removing it resulted in quick episode termination (after 250 steps) because the head was underwater for too long.

Overall, these results indicate that the reward components behave largely as intended and that the simulation environment is suitable for exploring swimming behaviour.

Reward Variation	Av. Vel.	% Head Above	Longest Streak	Description of Swimming Style
no continuous body punishment	0.4389	95.70%	41 steps	flying, fluttering head, pretty straight
no continuous body reward	0.8302	28.14%	101 steps	drowning, smooth movements, tail-first, head alternating between water surface
no continuous head reward	0.1795	65.94%	101 steps	pretty curled-up and upright, eventually drowning
no forward reward	0.0348	25.20%	54 steps	upright and nearly fully extended, fluttering movements
no head punishment	0.4680	86.90%	11 steps	tail-first, mainly using last body part for movement
no smoothness reward	0.6532	19.80%	90 steps	tail-first, fast movements, mostly under water
no survival reward	0.2422	47.60%	101 steps	drowning, body in u shape
no velocity punishment	0.4461	96.40%	29 steps	tail-first, partly flying

Table 3: Performance of the swimming worm after individually removing reward components from the baseline reward function.

3.1.2 Human

In the next step, we build upon the reward function developed for the worm agent and transfer the approach to a humanoid model with a comparable reward function structure. After thorough testing, we ended up with the following components and baseline values for our reward function (see Tab. 8 in App. A).

Unless specified otherwise we trained our agent for 2.5 million training steps. With the baseline parameters this resulted in the following performance measured for 1000 steps (see Tab. 4) :

Metric	Value
Average Forward Velocity	0.1529
Head Above Water:	5.68%
Longest streak of head below water	201 steps

Table 4: Our metrics applied to the humanoid, with baseline parameters. Max for head below water is 201 due to leniency setup.

We had a few iterations of testing whether we can improve our performance by adjusting our parameters. The most notable change here could be achieved by increasing the survival reward and disabling reward scaling. While this results in a swimming style that is a lot slower, it significantly improves the agents’ ability to keep its head above water (see Tab. 5) As our reward function is rather

Metric	Value
Average Forward Velocity	0.0550
Head Above Water:	89.00%
Longest streak of head below water	99 steps

Table 5: Our metrics applied to the humanoid, survival reward = 0.1, no reward scaling

complicated, we wanted to further explore the influence of it different compartments onto the swimmer

performance. For this we iteratively trained the agent, setting one reward component to zero for every iteration (see Tab. 6).

Reward Variation	Av. Vel.	% Head Above	Longest Streak	Description of Swimming Style
no body punishment	0.0159	66.50%	72 steps	"dog paddle"
no continuous body reward	0.0275	15.85%	201 steps	only leg kick, flies
no continuous head reward	0.0285	25.87%	201 steps	leg kick + rhythmic arm movement
no forward velocity reward	<i>-0.0358</i>	<i>3.83%</i>	<i>201 steps</i>	sinks, shakes in place
no head punishment	0.0846	100.00%	0 steps	big leg/arm strokes, flies
no pitch punishment	0.0560	100.00%	0 steps	bigger rhythmic stroke
no roll punishment	0.0773	98.00%	20 steps	rhythmic up/down strokes
no scaling	0.0738	98.40%	16 steps	rhythmic leg stroke, flies
no smoothness reward	0.0159	66.50%	72 steps	sinks, shaky movement
no survival reward	0.0421	88.20%	49 steps	Kicking leg movement, flies
no velocity punishment	0.0144	10.35%	201 steps	kicks under water, no forward movement

Table 6: Analysis of swimming styles based on reward modifications. Bold = across all metrics better performance than previously the best performance, italic for overall worse performance than baseline

Based on this, we can draw some inferences of what the effect of some rewards is. Just retracing our reward from its effect would lead to the following hypothesis: To achieve any forward movement both the forward velocity reward and the forward velocity punishment seem to be necessary. If we want to build our "keeping the head above water" condition on top of this, we need the continuous head reward and the continuous body reward. To also ensure that the rest of the swimmers body stays below the waterline the contributing reward factors seem to be more diverse: the continuous body reward, the head punishment, the reward scaling and the survival reward seem to have an impact. Rewards that seem to have become superfluous in the current configuration are the roll punishment and the pitch punishment. Similarly, the reward scaling does not seem to have a positive effect on the training. The body punishment seems to have an influence on the "head above water" condition and prevent flying, but lesser so than other reward components.

Metric	Value
Average Forward Velocity	-0.0081
Head Above Water:	93.90%
Longest streak of head below water	31 steps

Table 7: Our metrics applied to the humanoid, with baseline parameters, excluding punishment for roll/pitch and reward scaling

For a final assessment we thus trained the baseline policy with no roll and pitch punishment, and no reward scaling (see Tab. 7). While this looks a lot worse according to our metrics, the visualisation contradicts this (see App. A Fig. 5). In this case, the low velocity is due to the agent taking time to adjust its position to lie flat on the water, and starting its stroke later in the simulation.

3.2 LLM swimming

Our results at getting the LLM to swim were more mixed, and highly depended on the specific model used and the prompt itself. This mirrors the effects observed by Wang et al., 2024. To better quantify our results we tested five different prompt configurations with two different models (gemma 4 and gpt 4 mini), each predicting 50 actions:

- Setup + Setup + stroke explanation + Translation into Coordinates + incremental steps
- Setup + stroke explanation + translation into coordinates
- Setup + stroke explanation + translation into coordinates (limited to foot stroke)
- Setup + stroke explanation
- Setup

The results of our evaluation can be seen in App. C Tab. 9 for the gemma 4 model and Tab. 10 for the gpt 4 mini model. When focusing on our quantitative measures, the LLMs perform notably worse than our RL agent across all prompt variation. Given the differences in provided context and training time this is not overly surprising. When looking beyond the qualitative results we can observe further effects. Overall the model improves with a more detailed prompt, especially if it is aided in how to translate the conceptual movements into coordinates. As evident from the difference in performance between the "limited to foot stroke" vs the general modality, the LLMs both struggle execute the two movement patterns in parallel. Additionally, as can be seen from the interpolation modality where the LLM is asked to smaller sub-steps between the movement goals, the models fail to consistently infer the current movement direction from previous actions. What also can be observed across all prompts is a recognition of the arms and legs two distinct paired movement generators, which each have to be moved either in parallel or alternatingly to generate movement. What also should not be understated is that, while the resulting movement is not always as intended, both LLMs manage to perform an approximation of a backstroke movement with very little outside input.

4 Discussion

Overall, given the parameters we set for ourselves, we can say we broadly succeeded in our task to teach a humanoid to swim with different models. While our experimental set-up does not facilitate a detailed comparison of both learning styles, we can draw some general inferences from our observations. The fact that the RL agent learns directly from rewards in its environment is evident in its resulting swimming style. While it performs notably better at our set metrics its movements are more frantic and less coordinated. The LLMs struggle with keeping movements consistent across multiple actions and adjusting to (unexpected) outside influences, such as the body tilting towards the floor. But, they do seem to produce coordinated movement with very little outside input and can be comparatively easily directed to a certain action type. This combination of features suggests that a hybrid system, where the LLM provides seeding actions for the RL agent might produce interesting results. This could leverage the fast reactivity of a trained RL agent and still profit from the task outline the LLM can provide.

Beyond this, there are ample ways we could improve our experimental setup to makes this comparison more meaningful:

There is still room for us to improve upon our swimming simulation. The main caveat to our success is our environment. We neither model water as a proper fluid, nor do we model how the agent would interact with the different resistances by swimming at the water surface. While implementing this would not be trivial, it could be done by building a manual computation layer on the MuJoCo setup. We also currently use different starting XMLs for our RL agent and our LLM setup. While this would need a rework of our reward function it would be interesting to compare movements if we start the RL agent with a laying model.

For the LLM experiments, it would be a good next step to improve the control loop, for example, by incorporating torso stabilization to prevent the body from unstable rolling and flipping. A more stable controller could improve the execution of the pattern-generated swimming motion and, consequently, provide a more reliable foundation for LLM-based control. Furthermore, there is still room to reduce the response time of the LLM. One possible approach would be to finetuning a smaller model specifically for the swimming control task. A further avenue to improving the responses would be to add a translation layer between the LLM output and the JTC input, so the LLM can output natural language rather than coordinates.

In the RL experiments, our reward function could still be improved upon: a good follow-up would be to investigate if further simplifying the reward still leads to good results. Furthermore, for a more

realistic experiment setup, we could include energy expenditure in the reward. Outside of improving the reward, to follow our original idea more truly, it would be interesting to see what happens if we do not expect the agent to learn to swim completely from scratch. This could be achieved by integrating a CPG into the system or by starting training with an agent which is already capable of walking.

One of the main challenges we faced in this project was designing a suitable reward function such that the PPO agent learned the desired swimming behaviour. While exploring and testing reward components, we had the impression not all of them had the intended or expected effect. It took several iterations to develop a reward structure that produced consistent behaviour, while the agent frequently converged to local optima. Through this process, we gained a better understanding of reinforcement learning and the importance of careful reward design. In particular, we found that continuous reward components were generally more effective than abrupt, threshold-based rewards.

Another challenge was constructing the LLM prompt in such a way that the model produced trajectory positions in the required format and that these positions resulted in the intended swimming behaviour. It took several iterations to determine which prompt components were necessary and which provided the greatest benefit.

Overall, this project gave us valuable experience in simulation in MuJoCo and working with increasingly complex models with a larger number of degrees of freedom and joints. Throughout the project, we gained practical insights into reinforcement learning, reward engineering, controller design, and prompt engineering for LLM-based control. Beyond the technical aspects, it provided an interesting opportunity to approach the problem of humanoid swimming from multiple perspectives and to compare two fundamentally different control paradigms.

5 Conclusion

In this report, we explored two fundamentally different approaches for teaching a humanoid to swim: reinforcement learning and LLM-based control. We first developed a reinforcement learning framework based on PPO and investigated how different reward components influence the learned swimming behaviour. We then implemented a controller that allows an LLM to generate high-level swimming actions and evaluated the influence of different prompt designs on its performance.

Our reinforcement learning experiments show that a humanoid can learn to produce meaningful swimming motions in a simplified water environment. We showed that individual reward components have an influence on the resulting swimming style and that carefully balancing these rewards is essential for achieving stable swimming.

The LLM-based controller was also able to generate recognizable backstroke-like behaviour, although its performance remained below the RL agent.

Overall, our results demonstrated that both approaches are capable of producing swimming-like behaviour, with different strengths and limitations. While the RL approach is reactive and more conscious environmental metrics, LLM control generates a more classic swimming motion with comparably little task-specific training.

Appendix

A Reinforcement Learning

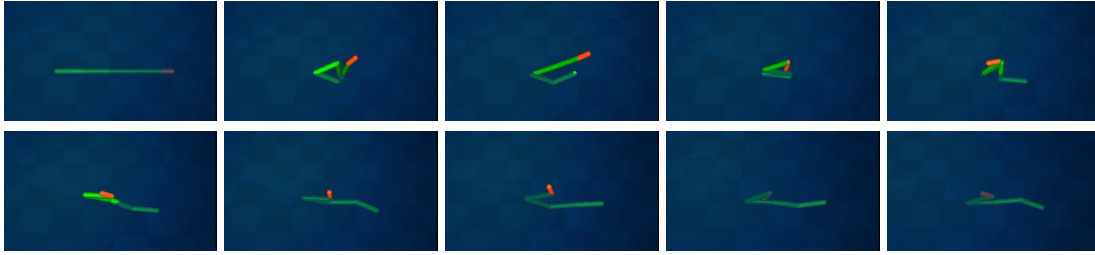


Figure 4: Baseline swimming behaviour of the worm agent. Consecutive frames are sampled at equal time intervals during one evaluation episode.

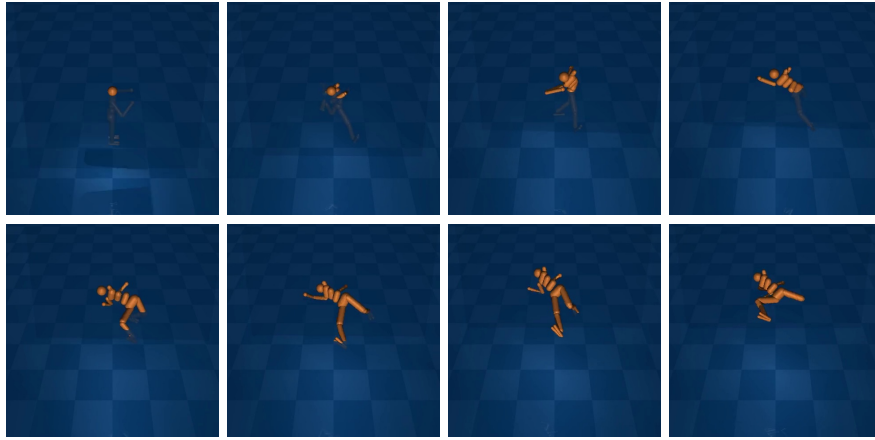


Figure 5: Optimized swimming behaviour of the humanoid agent. Consecutive frames are sampled at equal time intervals

Parameter Name	Reward Contribution	Baseline Scaling Value
Forward reward	$forward_velocity \times scaling_value$	40
Head punishment	If head underwater for 200 steps: $scaling_value$	5
Velocity punishment	If no forward velocity for 200 steps: $scaling_value$	1
Body punishment	If body above water after 1 Mil iterations: $scaling_value$	5
Smoothness reward	$difference_current_last_action \times scaling_value$	0.01
Continuous head reward	If head just above water: $(head_height - surface)^2 \times scaling_value$	5
Continuous body reward	If body below water: $(surface - body_height)^2 \times scaling_value$	5
Continuous body punishment	If limb above water: $(limb_height - surface - 0.5)^2 \times scaling_value$	5
Roll punishment	$roll_value \times scaling_value$	0.1
Pitch punishment	$pitch_value \times scaling_value$	0.5
Survival reward	$scaling_value$	0.01
Reward scaling	$total_reward \times scaling_value$	0.1

Table 8: Baseline values for the reward function components. Additional adjustable parameters: leniency - after how many steps we apply head and velocity punishment, Number of training steps after which we apply body punishment

B Pattern generated swimming



Figure 6: Procedurally generated backstroke pattern executed by the PD controller. Consecutive frames are sampled at equal time intervals.

C LLM swimming

Prompt Used	% Head Above Water	Forward Velocity	Description
Setup	30.00%	-0.0003	Small, parallel, bilateral Y axis movements of mainly the arms
Setup + stroke	78.00%	0.0094	starts out with alternating quarter rotation arm movements, then stops moving
Setup + stroke + translation (foot)	0.00%	-0.0579	Performs foot stroke as described, occasional small pauses, body tilts downwards, reducing effectiveness
Setup + stroke + translation	2.00%	-0.0330	Alternates correct foot stroke or arm stroke, arm stroke sometimes changes direction mid stroke
Setup + stroke + translation + interpolation	14.00%	0.0157	legs: parallel movement on Z axis, arms: attempt at alternating rotation, no consistent direction

Table 9: Results of different prompts for the gemma 4 model

Prompt Used	% Head Above Water	Forward Velocity	Description
Setup	8.00%	0.0163	Small, parallel, bilateral X/Z axis movements of mainly the arms
Setup + stroke	14.00%	0.0147	small alternating arm movements on mainly the Z axis
Setup + stroke + translation (foot)	0.00%	-0.0537	Performs foot stroke as described, occasional pauses, body tilts downwards, reducing effectiveness
Setup + stroke + translation	0.00%	0.0107	Mostly correct foot stroke, occasionally arm stroke, results into movement towards pool floor
Setup + stroke + translation + interpolation	0.00%	-0.0177	First correct leg movement, then only small back and forth rotations in the arms

Table 10: Results of different prompts for the gpt4 mini model

References

- Ahn, M., Brohan, A., Brown, N., Chebotar, Y., Cortes, O., David, B., Finn, C., Fu, C., Gopalakrishnan, K., Hausman, K., Herzog, A., Ho, D., Hsu, J., Ibarz, J., Ichter, B., Irpan, A., Jang, E., Ruano, R. J., Jeffrey, K., . . . Zeng, A. (2022). Do as i can, not as i say: Grounding language in robotic affordances. <https://arxiv.org/abs/2204.01691>
- Arjun, R. (2025, December). Teaching a humanoid robot to walk using reinforcement learning: A technical exploration with ppo and mujoco [Accessed: 2026-07-20]. <https://medium.com/@itsrarjun/teaching-a-humanoid-robot-to-walk-using-reinforcement-learning-a-technical-exploration-with-ppo-92e8a844740b>
- Chen, L., Lu, K., Rajeswaran, A., Lee, K., Grover, A., Laskin, M., Abbeel, P., Srinivas, A., & Mordatch, I. (2021). Decision transformer: Reinforcement learning via sequence modeling. *Advances in neural information processing systems*, 34, 15084–15097.
- Farama Foundation. (2024). Gymnasium swimmer environment documentation [Accessed: 2026-07-20].
- Google DeepMind. (2024). Mujoco humanoid model specification (XML) [Accessed: 2026-07-20].
- Heess, N., TB, D., Sriram, S., Lemmon, J., Merel, J., Wayne, G., Tassa, Y., Erez, T., Wang, Z., Eslami, S., et al. (2017). Emergence of locomotion behaviours in rich environments. arxiv 2017. *arXiv preprint arXiv:1707.02286*.
- MuJoCo. (2025). *Fluid forces* [MuJoCo Documentation]. Retrieved July 18, 2026, from <https://mujoco.readthedocs.io/en/stable/computation/fluid.html>
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. <https://arxiv.org/abs/1707.06347>
- Singh, R. P., Xie, Z., Gergondet, P., & Kanehiro, F. (2023). Learning bipedal walking for humanoids with current feedback. *IEEE Access*, 11, 82013–82023. <https://doi.org/10.1109/access.2023.3301175>
- Wang, Y.-J., Zhang, B., Chen, J., & Sreenath, K. (2024). Prompt a robot to walk with large language models. *2024 IEEE 63rd conference on decision and control (CDC)*, 1531–1538.
- Yu, W., Gileadi, N., Fu, C., Kirmani, S., Lee, K.-H., Arenas, M. G., Chiang, H.-T. L., Erez, T., Hasenclever, L., Humplik, J., et al. (2023). Language to rewards for robotic skill synthesis. *arXiv preprint arXiv:2306.08647*.